

ACH2118 — Introdução ao Processamento de Língua Natural

Exercício Prático 1 — Classificação de clareza de respostas do e-SIC

1. Nome e número USP

Integrante	Nº USP
Felipe Teixeira Rodrigues	13672213
Gabriel Fernandes Marchesano	15453229
Gabriel Alexo Rocha	13729636
Nicolas Garcia Santos	15481919

2. Estratégia adotada

O objetivo do trabalho é classificar automaticamente a clareza de respostas de sistemas públicos de informação em três classes (c1, c234 e c5). A estratégia final utiliza o **NorBERTo-base**, modelo desenvolvido pelo Itaú Unibanco e pré-treinado para a língua portuguesa. Baseado na arquitetura Transformer, o NorBERTo produz representações contextuais do texto, ou seja, interpreta cada palavra considerando também as palavras ao seu redor. Isso permite capturar relações semânticas e linguísticas que métodos baseados apenas em frequência de termos não representam diretamente.

Para adaptar o NorBERTo à tarefa foi utilizada a técnica **LoRA (Low-Rank Adaptation)**, criada como uma forma eficiente de realizar fine-tuning de modelos pré-treinados. Em vez de atualizar todos os parâmetros do modelo, o LoRA mantém a maior parte deles congelada e adiciona pequenas matrizes treináveis em determinadas camadas. Dessa forma, é possível especializar o modelo para a classificação de clareza com custo computacional e uso de memória significativamente menores do que em um fine-tuning completo.

Neste trabalho, o texto é tokenizado pelo próprio tokenizer do NorBERTo, com contexto máximo de **512 tokens** e truncamento que preserva o início e o final da resposta. Os adaptadores LoRA são aplicados às projeções de atenção Wqkv das 22 camadas do modelo, enquanto a cabeça de classificação de três classes também é treinada. A configuração utilizada na entrega emprega **rank 4, alpha 8 e dropout 0,05**, totalizando **272.643 parâmetros treináveis**.

Durante o desenvolvimento, observou-se que o espaço para melhoria apenas pela troca ou aumento da complexidade do modelo era provavelmente limitado pelas próprias informações disponíveis no conjunto de dados. Foram encontrados casos de **respostas textualmente idênticas associadas a rótulos diferentes**, indicando ruído ou subjetividade na rotulação. Além disso, a base disponibiliza somente o texto da resposta e sua classe: **a pergunta original feita pelo usuário não está disponível**, e muitas respostas fazem referência a **anexos, arquivos ou documentos cujo conteúdo também não está presente no dataset**. Assim, em parte dos exemplos, informações potencialmente importantes para avaliar a clareza não podem ser observadas pelo classificador, independentemente de sua complexidade.

O NorBERTo foi escolhido como modelo de entrega por representar uma abordagem mais moderna de PLN, baseada em um encoder contextual pré-treinado e adaptado de forma eficiente com LoRA, além de atender ao objetivo de explorar técnicas mais recentes no trabalho. Entretanto, os experimentos não demonstraram uma superioridade preditiva clara em relação a métodos tradicionais. Uma regressão logística baseada em TF-IDF de palavras e caracteres alcançou resultados muito próximos, com custo computacional e complexidade de implementação consideravelmente menores. Dessa forma, os resultados também mostram que, para este conjunto de dados, **modelos muito mais simples e leves podem atingir desempenho semelhante ao NorBERTo de forma mais direta**, enquanto o principal benefício do modelo contextual está na exploração de uma arquitetura mais moderna e semanticamente rica.

2.1. Trajetória de desenvolvimento

O desenvolvimento começou por métodos rápidos e progressivamente incorporou representações mais sofisticadas. Os resultados abaixo pertencem a rodadas e protocolos diferentes e, portanto, não devem ser interpretados como uma única tabela de ranking diretamente comparável; eles resumem as decisões que levaram ao modelo final.

Etapa	Estratégia	Principal conclusão
Baseline clássico	TF-IDF de palavras + regressão logística	Estabeleceu uma referência rápida e reproduzível para a tarefa.
Modelo lexical ampliado	TF-IDF de palavras e caracteres + regressão logística	Tornou-se uma referência forte; em uma das rodadas de três folds atingiu 45,14% de acurácia média externa.
LinearSVC	TF-IDF de palavras e/ou caracteres	Ficou muito próximo da regressão logística; a troca de classificador trouxe menos benefício que o ajuste da regularização.
Representações semânticas congeladas	Model2Vec/POTION e MiniLM, isolados ou combinados ao TF-IDF	Os embeddings trouxeram sinal adicional, mas não demonstraram vantagem consistente sobre a referência lexical.
MiniLM + MLP	Embeddings MiniLM congelados + MLP pequena	Melhorou sobre o controle semântico linear, mas ficou abaixo da referência lexical.
BERTimbau + LoRA	Encoder supervisionado em português	O aumento do volume de treino melhorou o resultado, mas o desempenho continuou próximo ou abaixo da referência lexical.
NorBERTo + LoRA	Encoder contextual pré-treinado com LoRA	Mostrou resultado promissor nos pilotos e foi levado a uma avaliação controlada com contexto de 512 tokens.
Diagnóstico e tratamento do dataset	Agrupamento, limpeza, normalização e hipóteses sobre duplicatas	As políticas testadas não apresentaram ganho estável; os textos e rótulos originais foram preservados.
NorBERTo-512 final	NorBERTo-base + LoRA, contexto 512	Tornou-se o modelo de entrega após CV no desenvolvimento e treino final nas 16.093 linhas de desenvolvimento.
Busca posterior de hiperparâmetros	Grid de learning_rate e rank LoRA	Houve melhora apenas marginal na CV e ela não se confirmou na avaliação posterior; o adapter anterior foi mantido.

Ao longo do desenvolvimento, foram testadas diferentes abordagens buscando uma relação adequada entre desempenho, complexidade e capacidade de representação, desde modelos mais simples baseados em atributos lexicais, como TF-IDF e regressão logística, até métodos com representações semânticas e modelos contextuais baseados em Transformers. Apesar do aumento

progressivo de complexidade, os modelos avaliados apresentaram resultados próximos e limitações semelhantes, sem que surgisse uma melhoria preditiva expressiva e consistente.

Esses resultados reforçaram a percepção de que parte relevante da dificuldade estava associada às próprias características da base, como inconsistências de rotulação e ausência de informações contextuais importantes. Diante dos ganhos cada vez menores obtidos com novas tentativas, optou-se por encerrar a exploração com o **NorBERTo adaptado por LoRA**, por representar a abordagem mais moderna entre as avaliadas e permitir investigar o uso de representações contextuais pré-treinadas sem o custo de um fine-tuning completo. A escolha, portanto, não implica que modelos mais complexos tenham demonstrado superioridade clara, mas representa o ponto final de uma sequência de experimentos em que abordagens mais simples já haviam alcançado desempenho semelhante.

3. Pré-processamento

Antes de definir o pré-processamento final, foram realizados experimentos específicos para verificar se uma limpeza mais agressiva do dataset poderia melhorar a classificação. A análise da base revelou alguns problemas relevantes: **9,83% das linhas pertencem a grupos de respostas exatamente repetidas que apresentam mais de um rótulo**, além de haver grande quantidade de respostas que fazem referência a anexos, documentos, links ou outras informações que não estão disponíveis para o modelo.

A partir desse diagnóstico, foram testadas diferentes estratégias de tratamento. Entre elas, foram avaliados o mascaramento de URLs, e-mails e identificadores, a exclusão de grupos com rótulos conflitantes durante o treinamento, a substituição do rótulo de respostas repetidas pelo voto majoritário, uma normalização mais ampla de identificadores, a inclusão de atributos estilísticos do texto, a remoção de exemplos considerados pouco consensuais entre diferentes classificadores e combinações dessas técnicas.

Nenhuma dessas intervenções produziu uma melhoria suficientemente consistente. Em uma primeira rodada, todas as estratégias de limpeza avaliadas ficaram abaixo do controle que utilizava os textos originais. Em uma segunda rodada, a melhor combinação apresentou ganho de apenas **0,16 ponto percentual**, correspondente a **25 acertos adicionais em 16.093 exemplos**, mas esse ganho não foi consistente entre os folds e o intervalo de comparação incluiu a possibilidade de não haver melhoria real.

Diante desse resultado, optou-se por **não modificar os textos nem os rótulos originais**. Embora algumas inconsistências possam indicar ruído na base, não há informação externa suficiente para determinar quais rótulos estariam incorretos. Alterá-los ou remover exemplos poderia eliminar informação válida e introduzir novas decisões arbitrárias no conjunto de treinamento. Como o benefício observado com a limpeza foi apenas marginal, considerou-se que o risco de perda ou distorção dos dados não justificava incorporar esses tratamentos ao pipeline final.

Assim, o pré-processamento utilizado pelo NorBERTo foi deliberadamente simples:

1. **Preservação de caixa:** maiúsculas e minúsculas são mantidas.

2. **Entidades HTML:** são convertidas para seus caracteres correspondentes.
3. **Espaços:** sequências de espaços em branco são reduzidas a um único espaço e espaços nas extremidades são removidos.
4. **Sem normalização linguística agressiva:** não são utilizados stemming, lematização, remoção de stopwords ou remoção de pontuação.
5. **Rótulos originais:** duplicatas conflitantes e casos ambíguos permanecem com os rótulos fornecidos na base.
6. **Tokenização:** utiliza-se o tokenizer original do NorBERTo-base.
7. **Comprimento máximo:** cada entrada possui no máximo **512 tokens**, incluindo tokens especiais.
8. **Truncamento head_tail:** quando o texto excede esse limite, preservam-se tokens do início e do final da resposta, descartando-se a região central.

Na partição de desenvolvimento com 16.093 respostas, a mediana foi de **178 tokens**, o percentil 95 de **701 tokens** e **1.744 respostas (10,84%)** ultrapassaram o limite de 512 tokens e precisaram ser truncadas.

É importante distinguir esse processamento da normalização utilizada apenas para construir os grupos de validação. Para impedir que respostas equivalentes aparecessem simultaneamente em treino e validação, textos semelhantes foram agrupados utilizando uma representação normalizada. Essa normalização, porém, **não foi aplicada ao texto fornecido ao NorBERTo**, que recebeu essencialmente o conteúdo original da resposta.

4. Parâmetros avaliados na busca em grade

Após a primeira versão do NorBERTo-512 já estar treinada e avaliada, foi realizada uma busca de hiperparâmetros adicional exclusivamente sobre as **16.093 linhas de desenvolvimento**, mantendo as 3.999 linhas do holdout fora da busca.

A busca concentrou-se nos dois hiperparâmetros considerados mais relevantes para a adaptação LoRA: a taxa de aprendizado e o rank dos adaptadores. O `lora_alpha` acompanhou o rank pela regra $\alpha = 2 \times \text{rank}$, mantendo constante a razão α/rank .

Parâmetro	Valores avaliados
<code>learning_rate</code>	{1e-4, 3e-4, 1e-3}
<code>lora_rank</code>	{4, 8, 16}
<code>lora_alpha</code>	{8, 16, 32}, com $\alpha = 2 \times \text{rank}$

Foram avaliadas **9 combinações**. Permaneceram fixos: contexto 512, truncamento `head_tail`, dropout LoRA 0,05, batch 4, acumulação de gradiente 4, weight decay 0,01, warmup 0,1, até três épocas, precisão FP32 e atenção SDPA.

O primeiro estágio utilizou um único split agrupado e fixo para screening. As três melhores configurações seguiram para uma CV separada de três folds, usando exatamente o protocolo de avaliação já implementado.

5. Valores obtidos e configuração final escolhida

A melhor configuração encontrada pela busca em grade foi o candidate-08, com $\text{learning_rate}=3\text{e-}4$, rank 16 e alpha 32. Entretanto, sua vantagem durante a validação cruzada foi pequena e não demonstrou ganho consistente em relação à configuração já utilizada.

Além disso, ao realizar o treinamento final dessa configuração, o comportamento observado não apresentou a mesma estabilidade do adapter anterior. Uma possível explicação está na diferença entre a seleção de checkpoints durante a validação cruzada e o treinamento final contínuo, realizado sobre toda a base e sem uma validação intermediária para selecionar automaticamente o melhor ponto da trajetória. Também foram observados sinais de maior instabilidade durante a otimização.

Investigar essas diferenças exigiria novas rodadas de treinamento, variações de seeds e novos experimentos de validação. Considerando o **alto custo computacional da busca em grade**, o ganho marginal observado e as próprias limitações do conjunto de dados e do exercício, optou-se por não ampliar novamente a busca.

Assim, o modelo final de entrega permaneceu sendo a configuração anteriormente encontrada, que já havia apresentado comportamento mais estável e resultados consistentes durante o desenvolvimento:

Parâmetro	Modelo final de entrega
learning_rate	1e-3
lora_rank	4
lora_alpha	8
lora_dropout	0,05
max_length	512
Truncamento	head_tail

Portanto, a busca por hiperparâmetros foi útil para avaliar configurações alternativas, mas **não demonstrou uma melhoria robusta o suficiente para justificar a substituição do modelo que já havia sido selecionado**.

6. Procedimento experimental

6.1. Separação dos dados

A base data/train.xlsx contém **20.092 respostas**. Após a etapa de diagnóstico, os dados foram separados em:

- **16.093 respostas de desenvolvimento;**
- **3.999 respostas de holdout fixo.**

A separação foi realizada considerando grupos de similaridade textual, de forma a reduzir o risco de respostas equivalentes ou muito próximas aparecerem simultaneamente em treino e avaliação.

O holdout foi mantido fora do treinamento, da seleção de checkpoints e da validação cruzada utilizada para comparar as configurações. Essa separação foi importante para evitar que decisões de modelagem fossem tomadas diretamente sobre o conjunto usado posteriormente para avaliação.

Durante o desenvolvimento também foram utilizados **hashes dos arquivos e das partições congeladas** para verificar que os mesmos dados e divisões estavam sendo reutilizados entre diferentes execuções, reduzindo o risco de alterações acidentais ou contaminação entre treino e avaliação.

6.2. Validação cruzada do NorBERTo-512

A avaliação do modelo escolhido utilizou **três folds externos agrupados** sobre as 16.093 linhas de desenvolvimento. A implementação permitia trabalhar com mais folds, mas foram utilizados três como compromisso entre robustez da avaliação e o elevado custo computacional do treinamento do NorBERTo.

Em cada rodada, um dos três folds era separado como **avaliação externa**. Esse conjunto permanecia fora do treinamento e da escolha do checkpoint, sendo utilizado apenas ao final para medir o desempenho do modelo naquela rodada.

Os dados restantes eram novamente divididos em duas partes:

- **treino do encoder:** utilizado efetivamente para atualizar os parâmetros do modelo;
- **validação interna:** utilizada durante o treinamento para acompanhar a acurácia e escolher o melhor checkpoint, com early stopping de paciência 1.

Dessa forma, a validação interna servia para decidir **em qual momento do treinamento o modelo deveria ser considerado pronto**, enquanto a avaliação externa media posteriormente como esse modelo se comportava em exemplos que não participaram dessa decisão.

O processo era repetido três vezes, alternando o fold externo. Assim, cada uma das 16.093 respostas aparecia exatamente uma vez na avaliação externa.

Os tamanhos foram:

Fold	Treino do encoder	Validação interna	Avaliação externa
1	8.572	2.199	5.322
2	8.769	2.176	5.148
3	8.151	2.319	5.623

Configuração utilizada:

Item	Valor
Modelo-base	Itau-Unibanco/NorBERTo-base
Contexto	512 tokens
LoRA	rank 4, alpha 8, dropout 0,05
Módulos LoRA	Wqkv das 22 camadas

Item	Valor
Parâmetros treináveis	272.643
Batch por dispositivo	4
Acumulação de gradiente	4
Batch efetivo nominal	16
Taxa de aprendizado	0,001
Weight decay	0,01
Warmup	10%
Máximo de épocas	3
Seleção do checkpoint	maior acurácia interna
Precisão	FP32
Atenção	SDPA
Seed-base	42

As melhores épocas selecionadas pela validação interna nos três folds foram **2, 3 e 2**. As métricas externas correspondentes foram então utilizadas para estimar o desempenho do modelo.

6.3. Treinamento do adapter final

Após a validação cruzada, os adapters dos folds foram utilizados apenas para avaliação. Para a entrega, um **novo modelo foi treinado do zero nas 16.093 respostas de desenvolvimento**, utilizando a configuração selecionada.

O número de épocas foi definido pela mediana das melhores épocas observadas nos folds, resultando em **duas épocas**.

Nesse treinamento final não foram utilizados holdout, early stopping ou seleção adicional de hiperparâmetros. O objetivo era utilizar toda a base de desenvolvimento disponível sem incorporar informação da avaliação final ao treinamento.

Após o treinamento, o adapter salvo foi recarregado e auditado para verificar a integridade do artefato antes da etapa de inferência.

6.4. Avaliação e geração da planilha de teste

A avaliação do holdout foi realizada em uma etapa separada, utilizando apenas o adapter já treinado, **sem qualquer atualização dos pesos do modelo**.

Posteriormente, o mesmo adapter foi utilizado para classificar as **900 respostas** de data/test1.xlsx. A planilha de teste não participou de treinamento, validação ou escolha de hiperparâmetros.

O arquivo original foi preservado e a versão final foi salva em:

deliveries/test1.xlsx

A auditoria final verificou que apenas a coluna clarity foi preenchida e que o restante da planilha permaneceu inalterado.

7. Resultado

O modelo final escolhido foi o **NorBERTo-base adaptado com LoRA**, utilizando rank 4, alpha 8 e contexto máximo de 512 tokens.

Modelo final	Conjunto de avaliação	Acurácia	F1 macro
NorBERTo-base + LoRA (rank 4, alpha 8), contexto 512	CV de 3 folds no desenvolvimento (16.093 predições OOF)	43,47%	43,22%
NorBERTo-base + LoRA (rank 4, alpha 8), contexto 512	Holdout fixo (3.999 respostas)	45,31%	45,31%

Na validação cruzada, a regressão logística baseada em TF-IDF utilizada como referência alcançou **43,99% de acurácia**, enquanto o NorBERTo obteve **43,47%**. No holdout, os resultados foram **45,31% para o NorBERTo** e **45,09% para a regressão logística**. As diferenças são pequenas e não permitem afirmar superioridade preditiva consistente de uma das abordagens.

No holdout, o NorBERTo apresentou recall de **41,85% para c1**, **42,70% para c234** e **51,40% para c5**. O comportamento por classe também reforça que diferentes modelos apresentaram vantagens pontuais, sem que uma abordagem dominasse de forma clara as demais.

Os resultados obtidos ao longo do desenvolvimento mostraram que o aumento da complexidade dos modelos não implicou necessariamente melhora de desempenho. Métodos lexicais relativamente simples permaneceram competitivos mesmo quando comparados a modelos contextuais pré-treinados. A exploração de diferentes técnicas de PLN foi, entretanto, importante para compreender na prática suas características, custos e limitações.

Também foram identificadas limitações relevantes na própria tarefa, como inconsistências de rotulação, respostas idênticas associadas a classes diferentes e ausência de informações potencialmente importantes para avaliar a clareza, especialmente a **pergunta original do usuário** e o conteúdo de **anexos ou documentos mencionados nas respostas**. Dessa forma, parte da informação necessária para uma classificação mais precisa não estava disponível aos modelos, limitando o ganho possível apenas por meio de algoritmos mais sofisticados.

A escolha final pelo **NorBERTo com LoRA** ocorreu principalmente pelo interesse em utilizar uma abordagem moderna de PLN, baseada em representações contextuais pré-treinadas e adaptação eficiente de parâmetros. Os experimentos indicam, porém, que modelos consideravelmente mais simples e leves podem alcançar resultados semelhantes neste conjunto de dados, com menor custo computacional e maior simplicidade de implementação.

A busca adicional por hiperparâmetros também apresentou apenas ganhos marginais durante a validação cruzada e não demonstrou a mesma estabilidade da configuração anteriormente encontrada. Considerando o elevado custo computacional de novas execuções e as limitações já observadas no problema, optou-se por manter como modelo de entrega a configuração **NorBERTo-512 com LoRA rank 4, alpha 8 e taxa de aprendizado 0,001**.

Por fim, a planilha oficial test1.xlsx não possui os rótulos verdadeiros. Portanto, a acurácia das **900 previsões entregues** somente poderá ser determinada após a disponibilização do gabarito.

8. Link para o repositório de código

<https://github.com/gabrielmarchesano/pln-ep1/>

O repositório contém o código de treinamento e inferência, configurações, documentação dos experimentos, resultados auditados e o adapter LoRA escolhido para entrega.

9. Instruções para reprodução

9.1. Ambiente

Recomenda-se Python 3.11 em ambiente virtual. As principais dependências versionadas são:

- transformers==4.57.1
- torch==2.8.0
- peft==0.17.1
- accelerate==1.10.1
- numpy==2.3.3
- pandas==2.3.2
- scikit-learn==1.7.2
- openpyxl==3.1.5

Criar o ambiente:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Para uso de GPU, a instalação do PyTorch deve ser compatível com o backend disponível (NVIDIA/CUDA ou AMD/ROCm). CPU também é suportada para inferência, porém é substancialmente mais lenta para treinamento.

9.2. Reproduzir a inferência da planilha de teste

O adapter final e o tokenizer já estão versionados no repositório. Não é necessário treinar novamente para gerar as previsões.

```
python src/main.py test \
--device auto \
--artifact-dir artifacts/norberto-lora-final-ctx512-20260928 \
--test data/test1.xlsx \
--output deliveries/test1-reproducao.xlsx \
--output-dir results/norberto/delivery/norberto-lora-test1-reproducao
```

Os pesos do NorBERTo-base não são armazenados no repositório; na primeira execução eles são baixados na revisão fixada caso não estejam em cache.

9.3. Reproduzir a validação cruzada do modelo escolhido

A partir da raiz do projeto:

```
PYTHONPATH=src python -m clarity.finetune experiment \
--train data/train.xlsx \
--config configs/norberto-lora-development-ctx512.json \
--holdout-splits results/research/dataset-hypotheses-20260923/splits.csv \
--output-dir results/norberto/delivery/norberto-lora-development-reproducao \
--artifact-dir artifacts/norberto-lora-development-reproducao
```

Devem ser utilizados diretórios novos, pois o pipeline recusa sobrescrever resultados existentes.

9.4. Treinar novamente um adapter final

Depois da CV, o comando público de treino pode gerar um novo adapter usando somente as linhas de desenvolvimento e mantendo o holdout excluído:

```
python src/main.py train \
--train data/train.xlsx \
--output-dir results/norberto/delivery/norberto-final-reproducao \
--artifact-dir artifacts/norberto-final-reproducao
```

Uma GPU CUDA ou ROCm é recomendada. O treino final original foi executado na AMD Radeon RX 6650 XT e levou aproximadamente 31 minutos.